



Introduzione a react

Programmazione

React è una libreria JavaScript per la costruzione di interfacce utente moderne, dinamiche e component-based.

Il suo obiettivo principale è semplificare la creazione di applicazioni web complesse suddividendo la UI in componenti riutilizzabili, ognuno responsabile di una parte specifica della pagina.



A differenza di un approccio tradizionale basato sulla manipolazione diretta del DOM, React utilizza un modello dichiarativo: lo sviluppatore descrive come l'interfaccia deve apparire in base allo stato dell'applicazione, e React si occupa di aggiornare in modo efficiente solo le parti necessarie della pagina.

Dal punto di vista tecnico, React lavora attraverso concetti fondamentali come componenti, props, state e rendering.



I componenti rappresentano blocchi autonomi dell'interfaccia, le props permettono di passare dati da un componente all'altro, mentre lo state gestisce le informazioni interne che possono cambiare nel tempo, come input utente, dati ricevuti da API o elementi visibili nella pagina.

Questo modello rende React particolarmente adatto allo sviluppo di Single Page Application, dashboard, gestionali, e-commerce e interfacce interattive, perché consente di mantenere il codice organizzato, modulare e più semplice da mantenere anche in progetti di grandi dimensioni.



Ecco gli elementi principali, spiegati in modo semplice e tecnico:

- **Componenti**

Sono i blocchi fondamentali di un'applicazione React. Ogni componente rappresenta una parte dell'interfaccia, ad esempio un menu, una card, un form, una tabella o una pagina intera. I componenti permettono di dividere la UI in parti più piccole, riutilizzabili e più facili da gestire.

- **JSX**

È una sintassi che permette di scrivere codice simile all'HTML direttamente dentro JavaScript. JSX rende più leggibile la costruzione dell'interfaccia, perché consente di descrivere la struttura visiva del componente insieme alla sua logica.



- **Props**

Le props sono dati passati da un componente padre a un componente figlio. Servono per rendere i componenti dinamici e riutilizzabili.

Per esempio, una stessa card può mostrare titoli, immagini o descrizioni diverse ricevendo valori differenti tramite props.

- **State**

Lo state rappresenta i dati interni di un componente che possono cambiare nel tempo.

Quando lo state cambia, React aggiorna automaticamente la parte dell'interfaccia coinvolta. È utile per gestire input, contatori, menu aperti o chiusi, dati caricati da API e interazioni dell'utente.



- **Rendering**

Il rendering è il processo con cui React trasforma componenti e dati in elementi visibili nella pagina.

React controlla quando l'interfaccia deve essere aggiornata e modifica solo le parti necessarie, rendendo l'applicazione più efficiente.

- **Virtual DOM**

È una rappresentazione leggera del DOM reale mantenuta da React in memoria.

Quando qualcosa cambia, React confronta la nuova versione con quella precedente e aggiorna solo gli elementi realmente modificati nel DOM della pagina.



- **Eventi**

React permette di gestire eventi dell'utente come click, submit, digitazione o selezione.

Gli eventi vengono collegati ai componenti tramite funzioni JavaScript, permettendo di rendere l'interfaccia interattiva.

- **Hooks**

Gli Hooks sono funzioni speciali che permettono di usare funzionalità avanzate nei componenti, come gestione dello stato, effetti collaterali, caricamento dati o accesso al ciclo di vita del componente. I più comuni sono useState e useEffect.

- **Lifecycle del componente**

Indica le fasi principali della vita di un componente: creazione, aggiornamento e rimozione dalla pagina.

In React moderno queste fasi vengono spesso gestite tramite gli Hooks, soprattutto con useEffect.



- **React Router**

È una libreria usata per gestire la navigazione tra pagine o viste diverse in una Single Page Application. Permette di creare percorsi come /home, /prodotti o /profilo senza ricaricare completamente la pagina.

- **Gestione dei dati da API**

React viene spesso usato per recuperare dati da server esterni tramite API REST o servizi backend. I dati ricevuti possono essere salvati nello state e mostrati dinamicamente nell'interfaccia.

- **Single Page Application**

Una SPA è un'applicazione web che carica una sola pagina iniziale e aggiorna dinamicamente il contenuto senza ricaricare tutto il sito. React è molto utilizzato per questo tipo di applicazioni perché rende fluida la navigazione e migliora l'esperienza utente.



- **Configurazione iniziale di React**

Per configurare un progetto React moderno si utilizza normalmente Node.js insieme a un build tool come Vite, che permette di creare rapidamente la struttura del progetto, avviare un server di sviluppo locale e preparare l'applicazione per la produzione.

La documentazione ufficiale React consiglia di partire da strumenti di build come Vite, Parcel o Rsbuild, mentre Vite mette a disposizione template già pronti per React e TypeScript.

1. `npm create vite@latest mio-progetto-react`
2. `cd mio-progetto-react`
3. `npm install`
4. `npm run dev`



Questo comando crea un nuovo progetto React, installa le dipendenze e avvia l'applicazione in locale; la cartella conterrà file come package.json, index.html, src/main.jsx e src/App.jsx.

- **Componente**

Un componente è un blocco riutilizzabile dell'interfaccia.

In React ogni parte della pagina può essere trasformata in un componente: un bottone, una card, un menu, una lista o un'intera schermata.

React permette di costruire interfacce combinando componenti più piccoli in strutture più grandi.

```
1.function Titolo() {  
2.  return <h1>Benvenuto in React</h1>;  
3.}  
4.  
5.export default Titolo;
```



- **JSX**

Il JSX è una sintassi che permette di scrivere una struttura simile all'HTML dentro JavaScript.

Non è HTML puro, ma una forma più leggibile per descrivere l'interfaccia grafica di un componente React.

```

1.function App() {
2.  const nome = "Mirko";
3.
4.  return (
5.    <div>
6.      <h1>Ciao {nome}</h1>
7.      <p>Questa è una pagina React.</p>
8.    </div>
9.  );
10.}
11.
12.export default App;
```



- **Props**

Le props sono dati passati da un componente padre a un componente figlio.

Servono per rendere i componenti dinamici e riutilizzabili, evitando di riscrivere lo stesso codice più volte con contenuti diversi.

```
1.function Card({ titolo, descrizione }) {  
2.  return (  
3.    <div>  
4.      <h2>{titolo}</h2>  
5.      <p>{descrizione}</p>  
6.    </div>  
7.  );  
8.}  
9.  
10.function App() {  
11.  return (  
12.    <Card  
13.      titolo="React"  
14.      descrizione="Libreria per creare interfacce utente."  
15.    />  
16.  );  
17.}  
18.  
19.export default App;
```



- **State con useState**

Lo state rappresenta un dato interno del componente che può cambiare nel tempo.

Con useState è possibile creare una variabile di stato e una funzione per aggiornarla; quando lo stato cambia, React aggiorna automaticamente la parte interessata dell'interfaccia.

```
1. import { useState } from "react";
2.
3. function Contatore() {
4.   const [numero, setNumero] = useState(0);
5.
6.   return (
7.     <div>
8.       <p>Valore: {numero}</p>
9.       <button onClick={() => setNumero(numero + 1)}>
10.        Aumenta
11.      </button>
12.    </div>
13.  );
14. }
15.
16. export default Contatore;
```



- **Eventi**

Gli eventi permettono di reagire alle azioni dell'utente, come click, digitazione, invio di form o selezione di elementi.

In React gli eventi vengono collegati direttamente agli elementi JSX tramite funzioni.

```
1.function BottoneSaluto() {  
2.  function saluta() {  
3.    alert("Ciao da React!");  
4.  }  
5.  
6.  return (  
7.    <button onClick={saluta}>  
8.      Clicca qui  
9.    </button>  
10. );  
11.}  
12.  
13.export default BottoneSaluto;
```



- **Rendering condizionale**

Il rendering condizionale permette di mostrare elementi diversi in base a una condizione.

È utile per gestire login, messaggi di errore, contenuti nascosti, schermate alternative o stati dell'interfaccia.

```

1.function Messaggio({ loggato }) {
2.  return (
3.    <div>
4.      {loggato ? (
5.        <p>Benvenuto utente!</p>
6.      ) : (
7.        <p>Effettua il login.</p>
8.      )}
9.    </div>
10. );
11.}
12.
13.export default Messaggio;
```



- Liste e map()

In React le liste vengono spesso generate partendo da array JavaScript. Il metodo map() permette di trasformare ogni elemento dell'array in un elemento JSX.

Ogni elemento generato deve avere una proprietà key, utile a React per identificare correttamente gli elementi della lista.

```

1.function ListaCorsi() {
2.  const corsi = ["React", "Angular", "Vue"];
3.
4.  return (
5.    <ul>
6.      {corsi.map((corso, index) => (
7.        <li key={index}>{corso}</li>
8.      ))}
9.    </ul>
10.  );
11.}
12.
13.export default ListaCorsi;
```



- **Form e input controllati**

Un input controllato è un campo di form il cui valore viene gestito dallo state di React.

Questo approccio permette di controllare, validare e utilizzare i dati inseriti dall'utente direttamente nel componente.

```
1. import { useState } from "react";
2.
3. function FormNome() {
4.   const [nome, setNome] = useState("");
5.
6.   return (
7.     <div>
8.       <input
9.         type="text"
10.        value={nome}
11.        onChange={(event) => setNome(event.target.value)}
12.        placeholder="Inserisci il nome"
13.      />
14.
15.      <p>Nome inserito: {nome}</p>
16.    </div>
17.  );
18. }
19.
20. export default FormNome;
```



- **Effetti con useEffect**

useEffect è un Hook usato per gestire operazioni che avvengono “fuori” dal normale rendering del componente, come chiamate API, timer, aggiornamenti del titolo della pagina o servizi esterni.

Le dipendenze indicate nell'array finale controllano quando l'effetto deve essere eseguito.

```
1. import { useEffect, useState } from "react";
2.
3. function Orologio() {
4.   const [secondi, setSecondi] = useState(0);
5.
6.   useEffect(() => {
7.     const timer = setInterval(() => {
8.       setSecondi((valore) => valore + 1);
9.     }, 1000);
10.
11.    return () => clearInterval(timer);
12.  }, []);
13.
14.  return <p>Secondi trascorsi: {secondi}</p>;
15.}
16.
17. export default Orologio;
```



- **Chiamata API**

React viene spesso usato per recuperare dati da un backend o da un servizio esterno.

Una chiamata API viene gestita dentro useEffect, mentre i dati ricevuti vengono salvati nello state e poi visti nell'interfaccia.

```
1. import { useEffect, useState } from "react";
2.
3. function Utenti() {
4.   const [utenti, setUtenti] = useState([]);
5.
6.   useEffect(() => {
7.     fetch("https://jsonplaceholder.typicode.com/users")
8.       .then((response) => response.json())
9.       .then((data) => setUtenti(data));
10.  }, []);
11.
12.  return (
13.    <ul>
14.      {utenti.map((utente) => (
15.        <li key={utente.id}>{utente.name}</li>
16.      ))}
17.    </ul>
18.  );
19. }
20.
21. export default Utenti;
```



Le problematiche comuni nello sviluppo con React riguardano soprattutto la gestione dello state, la comunicazione tra componenti e l'organizzazione del progetto.

Nei progetti semplici React risulta immediato, ma quando l'applicazione cresce possono comparire difficoltà legate a componenti troppo grandi, props passate attraverso troppi livelli, rendering inutili, chiamate API ripetute o gestione disordinata degli effetti con useEffect.

Un altro problema frequente è confondere React con un framework completo: React si occupa principalmente della UI, quindi per routing, gestione avanzata dello stato, validazione, autenticazione o chiamate al backend spesso servono librerie aggiuntive.



Per questo è importante progettare bene la struttura dei componenti, separare la logica dalla parte grafica, usare correttamente gli Hooks e mantenere il codice modulare.

Buon Davante a tutti

